

MC read data-return path

One channel, DDR5. DFI read capture → accumulate → RD_SRAM → completion. All ports + timing explained. Widths parametric (DQ, N_RANKS, 512=line). Mirror of the write launch, but **PHY-timed**, not self-timed.

1 Shape of a read (vs a write)

A RD command is issued on the CA bus; the data comes back **RL = CL** cycles later at the DRAM pins. The MC does *not* generate the data timing — the **PHY drives dfi_rddata** with a **dfi_rddata_valid** strobe when the beats are actually present. The MC's job is:

- at RD issue, **arm** the capture: assert **dfi_rddata_en** at **cmd + tphy_rdlat** (opens the capture window for BL/2 beats), and push {**dbuf_addr**, **rob_index**} into a fixed-depth **RL delay line** so the destination is known when data lands;
- **capture** beats on **dfi_rddata_valid**, accumulate into a 512b line, write *once* to RD_SRAM at **dbuf_addr**;
- on the last beat, raise **rd_done**; the ROB marks its bitmap; when all packets of a request are in, **MERGE** drains RD_SRAM to the AXI R channel in order.

Key contrast with the write path: write data is **self-timed** (MC drives it out); read data is **PHY-timed** (MC waits for **dfi_rddata_valid**). There is no “en leads data” offset the MC owns — the MC opens a window (**rddata_en**) and the PHY fills it.

2 DFI read-data ports

port	dir	width	meaning / when
dfi_rddata_pN	PHY→MC	2*DQ	read beats for phase N (2 UI). Valid only when rddata_valid_pN=1 .
dfi_rddata_valid_pN	PHY→MC	1	PHY asserts when dfi_rddata_pN carries valid beats. The capture strobe.
dfi_rddata_en_pN	MC→PHY	1	MC opens the capture window: assert at cmd+tphy_rdlat , hold BL/2. Tells PHY to return data.
dfi_rddata_dbi_pN	PHY→MC	2*DQ/8	read DBI (if enabled); PHY may pre-resolve. Optional.
dfi_rddata_cs_n / valid variants	MC→PHY	few	some PHYs qualify per-rank; PHY-IP dependent (parametric).

Read CRC / on-die ECC: handled by DRAM/PHY, not the MC data path (same as write — MC only monitors **dfi_alert_n** / **dfi_error**). No CRC/ECC data port the MC drives.

3 Timing

- **tphy_rdlat** — delay (DFI cycles) from the RD command to when the MC asserts **dfi_rddata_en** (open the window). PHY-reported, DVFS-reloadable, **same across ranks** (per-rank skew absorbed in the PHY, invisible on DFI).
- **RL = CL** — command-to-data at the DRAM pins. The PHY maps the MC's window to the pin timing; the MC counts only DFI cycles.
- **Capture window** — **dfi_rddata_en** held for BL/2 DFI-beats; PHY returns **dfi_rddata** + **dfi_rddata_valid** within it. The MC keys capture off **rddata_valid** (robust to small PHY skew), not a fixed offset.
- **RL delay line** — deterministic depth; loaded with {**dbuf_addr**, **rob_index**} at RD issue, pops at the capture point so the accumulator knows where to write. No CAM, no timestamp.
- **dqFree** — read bursts occupy the DQ bus BL/2 = 8 CK each, same as writes; spacing / cross-rank tRTRS rules are identical to the write side (scoreboard).

4 Gear / beat placement

BL16 = 16 UI = 8 CK of data; a 64B line = 8 beat-groups of 8B (2 UI × DQ). Per **mc_clk** the MC captures 2***gear**

	gear	phases	mc_clks	byte capture per mc_clk
beats across the active phases (mirror of write split):	1:1	p0	8	p0: B0-7, then B8-15, ... B56-63 (one beat-gr
	1:2	p0,p1	4	cyc0 p0=B0-7 p1=B8-15 ; cyc1 p0=B16-23 p
	1:4	p0..p3	2	cyc0 p0..p3=B0-31 ; cyc1 p0..p3=B32-63

RD_ACCUM is the width/rate matcher (knows gear); **RD_SRAM** stays gear-agnostic — one wide 512b write per packet, not per beat.

5 Modules (read path)

module	role + I/O
RD_CAP	RL delay line + window arm. IN (at RD issue): dbuf_addr, rob_index, tphy_rdlat; dfi_rddata_valid. OUT: dfi_rddata_en, cap_en, accum_load, rd_done{rob_index,pkt}.
RD_ACCUM	beats → 512b. IN: dfi_rddata_pN[2*DQ], cap_en, gear. OUT: line[512], rd_wr_en, dbuf_addr.
RD_SRAM	512b/line store. IN: line[512] @ dbuf_addr (MC port). OUT: 512b read @ dbuf_addr (CIF/merge port). Dual-domain.
RD_SPLIT / MERGE	512b → AXI R beats, in segment order once the ROB bitmap is full. OUT: {rdata,rresp,rlast,rid} valid/ready.
COMPLETION	rd_done → ROB bitmap set; bitmap full → trigger MERGE; free dbuf range (RD_SRAM_MGR).

6 Points to get right (for your diagram)

- Read is **PHY-timed**: capture on **dfi_rddata_valid**; don't assume a fixed en→data offset the MC controls (that was the write side).
- **dfi_rddata_en** = window the MC opens (cmd+tphy_rdlat, hold BL/2); **rddata_valid** = PHY says data is here. Two different signals, two owners.
- **One wide 512b write** to **RD_SRAM** per packet (**RD_ACCUM** gathers first); not per-beat.
- **rd_done** = **last beat committed to RD_SRAM**, never before — completion can't fire until the final beat is in.
- **dbuf_addr** + **rob_index** ride the **RL delay line** from RD issue; no CAM.
- Multiple reads in flight: RL line depth $\approx$ RL/spacing (like the write WL line) — a **delay line**, not one counter (same reasoning as the write path).
- **tphy_rdlat** **same across ranks**; gear only rescales the DFI-cycle unit + capture width.